

genproc: Robust, Logged and Reproducible Iteration in R

Daniel Rakotomalala

Independent developer; author of the `genproc` and `scrutr` CRAN packages
rakdanielh@gmail.com

ISC grant programme, round 2026-2, revised submission with the budget within the 10 000 USD limit. Source: https://github.com/danielrak/genproc_isc-proposal.

Executive Summary

When a procedure runs over many cases (files, regions, subjects, parameter sets) and one case fails, the R user needs to see which case failed and why, re-run that case only, and keep the record of the original run. When an input file changes, the user needs to know which cases depend on it. `genproc`, on CRAN since May 2026, gives this with one call. The user supplies a function and a *mask*, a `data.frame` with one row per case. `genproc()` runs the function on every row and returns one object: a log with one row per case and the real traceback of each failure; a reproducibility snapshot (R and package versions, the exact mask, a size-and-date fingerprint of every input file named in the mask); and helpers that compare two runs and re-run only the cases that failed or whose inputs changed. Parallel execution (on `future`), non-blocking execution and progress reporting (on `progressr`) are optional layers that compose on top.

```
r <- genproc(convert, mask, parallel = parallel_spec(workers = 4))
errors(r)                # the failed cases, with their tracebacks
diff_inputs(r_previous, r) # which input files changed since the last run
rerun_failed(r, convert)  # re-run the failed cases only
```

Version 0.2.0 ships all of this with 13 exported functions, close to 300 test blocks, continuous integration on Linux, macOS and Windows, and a documentation site. Its result object follows one compatibility rule: fields are added, never removed or renamed.

The ambition goes beyond one package: a written, versioned contract for iterated runs in R (mask, result, errors, retries, reproducibility, monitoring), published for review and proposed to other package authors, with `genproc` as its reference implementation. Henrik Bengtsson, author of `future` and the R Foundation's representative on the ISC, commented in detail on the outline in April 2026 and saw the same need; his advice shaped the plan below.

The grant funds twelve months of work, January to December 2027, in five milestones. Each funded milestone leaves a documented, tested and usable improvement: (1) the written contract, including the error-tolerance and retry rules; (2) a backwards-compatible `genproc_mask` class that implements it; (3) error replay, content-hash input fingerprints and live monitoring of non-blocking runs; (4) evaluation by outside users and a worked `targets` integration; (5) a 1.0.0 release with a stable lifecycle and a succession process.

Budget: 10 000 USD, all labour, itemised by milestone. If the ISC funds only the first three milestones, the community still gets the contract and a package that is strictly more capable than today.

Signatories

Project team

Daniel Rakotomalala (proposer, sole developer). Author and maintainer of `genproc` and of `scrutr` (data validation), both on CRAN. PhD in economics. Data scientist and project lead in a national statistical service, where the pipelines that motivated `genproc` run daily. Skills: R package development (S3, non-standard evaluation, AST rewriting), the `future` ecosystem, CI/CD, technical writing. Project site: <https://danielrak.github.io/genproc/>. He works on `genproc` outside a full-time job; the 200 hours below are about four hours a week.

The proposer uses AI coding assistants (large language models) to draft code, tests and text, including parts of this document. He designs, reviews every change, tests the package by hand and takes every decision; the code is kept readable so that it can be maintained without such tools.

Contributors

None at submission; the work is sized for one developer. Contributors will be invited from Milestone 1 (contract review) and Milestone 4 (evaluation).

Consulted

- **Henrik Bengtsson**, author of `future`, `future.apply` and `progressr`, on which the parallel, non-blocking and monitoring layers are built, and the R Foundation’s representative on the ISC. Contacted in April 2026; he replied in detail. He noted that error handling in `apply()`-style loops is a frequent user question and, on the whole, saw these topics as ones the R community would benefit from standardising, for sequential as well as parallel processing. Three specific points shaped this proposal: define the retry rules for sequential processing first, then carry them to parallel processing; use `progressr` conditions for monitoring (done in 0.2.0) and extend them to non-blocking runs (Milestone 3); stay above `future::plan()` and leave backend negotiation to the futureverse.
- **William Michael Landau**, author of `targets`. Contacted in April 2026 on the positioning of `genproc` relative to `targets`. No reply at the time of writing.

No other ISC member has been consulted. Feedback from R users in public statistics and clinical data will be sought in the start-up phase, through the Milestone 1 review.

The Problem

What the problem is

R users routinely write a procedure that works on one case (one file, one region, one subject, one parameter set) and then need to run it on many. The step from “a script that works once” to “a run over a hundred cases” is usually improvised: a `for` loop or `lapply()` with an ad hoc `tryCatch()`, some `future` code added for speed, and no record of which package versions were loaded or which input files were read. When one case fails, the run often stops, or the failure goes unnoticed. When an upstream file is rewritten between two runs, nothing flags it. When someone asks what exactly was run, the answer is reconstructed from memory.

Who it affects

Anyone who applies the same procedure to many entities. The primary setting, and the proposer’s own practice, is file processing in public statistics and recurring reporting: the same processing applied to every region, or every month over the same paths, where the run must be auditable (which code, which inputs, which cases failed and why) and a silently modified input file is a common cause of a wrong number reaching a decision maker. The same pattern appears in clinical data pipelines (one derivation per study or site, with traceback and environment capture required for review) and in simulation studies (thousands of parameter sets, where one failure must not cost the others).

Why it should be solved

Each team pays the same improvisation cost on its own, and the code it produces is rarely reusable or auditable by anyone else. Error handling in `apply()`-style loops is a frequent user question (see Consulted). Codifying the pattern once, behind a stable contract, lowers that cost for the whole community, as `testthat` did for testing. The aim is a proposed, versioned contract for iterated runs in R, with `genproc` as its reference implementation.

Existing work

Every ingredient exists. `purrr` (Wickham and Henry 2026) gives `safely()` and `insistently()` (retries); `furrr` (Vaughan and Dancho 2026) and `future.apply` (Bengtsson 2021, 2026a) map in parallel on `future`; `progressr` (Bengtsson 2026b) signals progress. `BiocParallel` (Morgan et al. 2026) flags failed elements (`bpok()`), re-runs only those (`BPREDO`) and logs tracebacks. `batchtools` (Lang et al. 2017) keeps a job registry that can be inspected and resumed. `targets` (Landau 2021) branches over rows and files, saves workspaces for debugging, and manages a whole project’s dependencies; `genproc` is designed to run inside a target, not instead of one.

What none of them offers for a standalone batch procedure is one call on a function and a table that returns, in one object, the per-case log with tracebacks, the environment and input-file snapshot, and the means to re-run only what failed or changed, with no job registry to set up and no pipeline to restructure. **genproc** (Rakotomalala 2026) returns that object today. It targets procedures that write their output per case (a file, a table, a database row); collecting in-memory return values is out of scope. Whether this interface saves implementation and recovery work is the claim that Milestone 4 tests with outside users. What is missing today: the written contract behind the mask, the layers that make failed cases replayable and long runs observable, evidence from users other than the author, and the stewardship that lets the package outlive him.

The proposal

Overview

The grant takes **genproc** from an experimental 0.2.0 to a stable 1.0.0 in twelve months, January to December 2027, in five milestones:

1. **Write the contract.** The package relies on an implicit model: a *unit* is a mask row, a *parameter* a mask column, and operations are independent across units. Milestone 1 writes it down as a versioned vignette and glossary, and defines the error-tolerance and retry rules for sequential runs first, then for parallel runs: what a retry is, when it is allowed, how it is logged, and what counts as a handled error as opposed to an interruption, a worker failure or a partly written output. It adds two concepts, shown on a file-conversion mask with columns `src_dir`, `src_file` and `dst_file`. A *parameter dependency* says that `dst_file` is computed from `src_file` (same name, new extension), so it is derived rather than typed and cannot drift out of step. An *input group* says that `src_dir` and `src_file` together name one input file, so fingerprints and `diff_inputs()` follow that file rather than two unrelated strings. Published for public review before dependent code is merged.
2. **Implement it in the mask.** A `genproc_mask` S3 class on which the user declares dependencies and input groups, and which gives each case an identity based on its free parameters, so that reordering rows does not change it and repeated rows are told apart. Every existing `data.frame` mask keeps working unchanged.
3. **Finish the execution layers.** Error replay, content-hash input fingerprints and live progress for non-blocking runs, each with its supported backends documented.
4. **Test it with others.** Outside users run documented failure-and-recovery tasks on release 0.4.0 and their feedback is published; a worked example shows a **genproc** run as one step of a **targets** pipeline.
5. **Prepare the hand-over.** Release 1.0.0 with lifecycle “stable”, a maintainer guide, a release checklist, documented design decisions and a succession process.

A Shiny proof of concept (<https://github.com/danielrak/genprocShiny>) stays outside the grant; it will be rebuilt on the public API later.

Benefits at three levels: practitioners get a ready-made contract instead of hand-written scaffolding; package authors get a mask and result API to build on; the ecosystem gets a written, reviewed contract for what a robust iterated run is.

Detail

Minimum Viable Product

The MVP already exists: **genproc** 0.2.0 on CRAN, with the five layers described under Architecture, result contract, tests and documentation. Each milestone adds one reviewable artefact in a tagged release.

Architecture

genproc() wraps an iteration kernel in orthogonal layers: *logged* and *reproducibility* are always on; *parallel* (`future.apply`), *non-blocking* (a background future polled with `status()` and collected with `await()`) and *monitoring* (`progressr` conditions) are opt-in and compose. A new layer is an extra argument to **genproc()** and extra fields on `genproc_result`.

Assumptions

- The **future** framework remains the standard for R parallelism. **genproc** stays above `plan()` and does not negotiate resources itself, so the backend negotiation the futureverse plans is an opportunity, not a conflict.
- A backwards-compatible generalisation of the mask is feasible. The 0.2.0 test suite covers every documented mask pattern and gates every Milestone 2 change.

- One contract serves the settings of the Problem section; the Milestone 1 review and the Milestone 4 evaluation test this.

Failure modes and their handling. The sole developer is unavailable for a period: milestones are ordered by value, the ISC is told in the next monthly note, and a revised schedule or a reduced scope is agreed. `genproc_mask` breaks an existing usage: reverted and redesigned behind the compatibility suite before release. Too few outside users for Milestone 4: the tasks are run with the proposer’s own team and the report says so. No co-maintainer by Milestone 5: 1.0.0 ships, the proposer remains maintainer until a hand-over is accepted, and the closing report says so.

External dependencies

Runtime imports stay `future`, `future.apply`, `progressr` and base R. The grant adds no hard dependency. Optional additions: `digest` or `rlang`’s hashing for content-hash fingerprints; `targets` in `Suggests` for the integration vignette.

Project plan

Start-up phase

Nothing needs to be built to start. The public repository <https://github.com/danielrak/genproc> already has an issue tracker, R CMD check on Linux, macOS and Windows, code coverage, and automatic deployment of the documentation site. In the first month a `CONTRIBUTING.md`, a code of conduct and a “Milestone 1 review” GitHub discussion are added, R users in public statistics and clinical data are invited to that review, and the coverage baseline is recorded. Reporting: a monthly note to the ISC, a `NEWS.md` entry and a tagged release at every milestone close.

Technical delivery

Dates assume a contract signed by December 2026 and work from January 2027.

M1, by 2027-03-31. Contract. The vignette and glossary described above. Published for review; comments resolved before M2 code is merged.

M2, by 2027-05-31. `genproc_mask`. S3 class with declared parameter dependencies, input groups and a content-based `case_id`. Constructor, validator, print method, tests for every documented mask pattern. Release 0.3.0 tagged and submitted to CRAN.

M3, by 2027-09-15. Execution layers. `replay(result, case_id)` re-runs one failed case from its recorded arguments and reports any difference in inputs and package versions since the original run (rebuilding past software environments is out of scope); `method = "hash"` for content-hash input fingerprints; live progress of non-blocking runs, collected in the background and queried on demand, with a table of what each backend supports. Release 0.4.0 tagged and submitted to CRAN.

M4, by 2027-11-15. Evaluation and integration. At least three users outside the project run the documented tasks on 0.4.0: find a failed case, read its traceback, re-run it, detect a changed input and re-run the affected cases. What they could and could not do is published and fixed where needed. A vignette shows a `genproc` run as one step of a `targets` pipeline, with declared file dependencies and the pipeline’s behaviour when some cases fail.

M5, by 2027-12-31. 1.0.0 and hand-over. Lifecycle set to stable; maintainer guide, release checklist, design decisions and succession process; a prospective co-maintainer recruited among reviewers and evaluators and mentored through one release; closing post on the R Consortium blog.

Other aspects

MIT code; CC BY 4.0 documentation (notice added in the first month); public GitHub hosting during and after the grant. Communication: an announcement post, one R Consortium blog post per quarter and a delivery post; a talk submitted to user! 2027; posts on social media and R-bloggers at each release.

Budget & funding plan

Total **10 000 USD**, labour only, 200 hours at 50 USD per hour; no travel, hardware or cloud costs. The amounts allow partial funding in the order listed (50% at signature, 50% at completion).

Mile-stone	Deliverable	Hours	USD
M1	Contract vignette, glossary, retry rules, public review	40	2 000
M2	genproc_mask class, compatibility suite, release 0.3.0	40	2 000
M3	Replay, hash fingerprints, live progress, release 0.4.0	60	3 000
M4	Evaluation by outside users, targets vignette	30	1 500
M5	Release 1.0.0, governance, succession process, closing post	30	1 500

Success

Definition of done

- **M1**: contract vignette on the documentation site, review discussion closed and comments answered.
- **M2**: 0.3.0 tagged and submitted to CRAN with **genproc_mask**, the pre-grant test suite passing unchanged.
- **M3**: 0.4.0 tagged and submitted to CRAN with replay, hash fingerprints and live non-blocking progress.
- **M4**: evaluation report published; **targets** vignette on the site.
- **M5**: 1.0.0 tagged and submitted to CRAN, lifecycle stable, governance documents merged, closing post published.

Measuring success

- Acceptance tests in the repository: a failed case is identified, unaffected cases complete, a targeted re-run stays linked to the original attempt, reordering the mask keeps case identity, a changed input selects the expected cases.
- Coverage at or above the recorded baseline; zero regressions on the compatibility suite.
- Outside users complete the Milestone 4 tasks without help from the proposer; their reports are public.
- Issues and pull requests from others; at least one package or public project using **genproc** by the end of the grant.
- A co-maintainer who has reviewed changes and prepared one release; if none, the closing report says who maintains the package.

Future work

Cancellation of non-blocking runs where the backend allows it; a `tar_genproc()` helper for **targets** pipelines; a checker that compares declared dependencies and input groups with what the function actually reads; **genprocShiny** rebuilt on the public API; a software paper for the R Journal. The proposer remains maintainer after the grant until a hand-over is accepted; these directions are decided with the co-maintainer and the community.

Bengtsson, Henrik. 2021. "A Unifying Framework for Parallel and Distributed Processing in R Using Futures." *The R Journal* 13 (2): 208–27. <https://doi.org/10.32614/RJ-2021-048>.

Bengtsson, Henrik. 2026a. *future.apply: Apply Function to Elements in Parallel Using Futures*. <https://CRAN.R-project.org/package=future.apply>.

Bengtsson, Henrik. 2026b. *progressr: An Inclusive, Unifying API for Progress Updates*. <https://CRAN.R-project.org/package=progressr>.

Landau, William Michael. 2021. "The targets R Package: A Dynamic Make-Like Function-Oriented Pipeline Toolkit for Reproducibility and High-Performance Computing." *Journal of Open Source Software* 6 (57): 2959. <https://doi.org/10.21105/joss.02959>.

Lang, Michel, Bernd Bischl, and Dirk Surmann. 2017. "batchtools: Tools for R to Work on Batch Systems." *Journal of Open Source Software* 2 (10): 135. <https://doi.org/10.21105/joss.00135>.

Morgan, Martin, Jiefei Wang, Valerie Obenchain, Michel Lang, Ryan Thompson, and Nitesh Turaga. 2026. *BiocParallel: Bioconductor Facilities for Parallel Evaluation*. <https://doi.org/10.18129/B9.bioc.BiocParallel>.

Rakotomalala, Daniel. 2026. *genproc: Robust, Logged and Reproducible Iteration at Organizational Scale*. <https://CRAN.R-project.org/package=genproc>.

Vaughan, Davis, and Matt Dancho. 2026. *furrr: Apply Mapping Functions in Parallel Using Futures*. <https://CRAN.R-project.org/package=furrr>.

Wickham, Hadley, and Lionel Henry. 2026. *purrr: Functional Programming Tools*. <https://CRAN.R-project.org/package=purrr>.